Propuestas de aplicaciones web

Integrantes del equipo: José Montoro, Fernando Ortega, Samuel Gallego

**IDEA 1: AutoTaller Cloud**

*Problema o necesidad*

Muchos talleres mecánicos pequeños siguen utilizando programas antiguos, hojas de cálculo o varias aplicaciones diferentes para organizar las citas, controlar las reparaciones, gestionar los recambios y preparar las facturas. Esto puede provocar errores, pérdida de información y dificultades para mantener informado al cliente.

*Usuarios destinatarios*

- Pequeños talleres mecánicos multimarca.
- Dueños y encargados de talleres.
- Mecánicos y trabajadores de recepción.
- Clientes que llevan sus vehículos al taller.

*Solución propuesta*

AutoTaller Cloud sería una aplicación web para gestionar desde un mismo sitio el funcionamiento diario de un taller. Permitiría guardar la información de los clientes y sus vehículos, organizar las citas, controlar las reparaciones y avisar al cliente sobre el estado de su vehículo.

La integración con VeriFactu y con proveedores de recambios se podría dejar preparada o simular con datos de prueba, ya que hacer una integración real podría ser demasiado complicado para el proyecto.

*Funcionalidades principales*

- Gestión de clientes y vehículos: Se podrán guardar los datos de los clientes y de sus vehículos, como la matrícula, la marca, el modelo, el kilometraje y el historial de reparaciones.
- Agenda de citas: El taller podrá crear, modificar y cancelar citas indicando el cliente, el vehículo, el motivo de la visita y el mecánico encargado.
- Órdenes de reparación: Se podrán registrar las averías, las reparaciones realizadas, las horas de trabajo, los recambios utilizados y el estado de cada reparación.
- Inspección digital del vehículo: El mecánico podrá completar una lista de comprobación y añadir fotografías de los daños o problemas encontrados. El cliente podrá consultar esta información mediante un enlace.

- Presupuestos y facturas: La aplicación podrá calcular el precio de la mano de obra y de los recambios, además de generar presupuestos y facturas de prueba.
- Avisos de mantenimiento: El sistema podrá enviar recordatorios sobre revisiones, cambios de aceite, ITV u otras operaciones de mantenimiento.

*Tecnologías que se podrían utilizar*

- HTML, CSS y JavaScript.
- Java con Spring Boot para el servidor.
- MySQL o PostgreSQL para la base de datos.
- API REST para conectar el frontend con el backend.
- Bootstrap para adaptar la aplicación a móviles y ordenadores.
- Git y GitHub para guardar diferentes versiones del proyecto.
- Docker para facilitar la instalación.
- Una biblioteca de Java para generar documentos PDF para los

**IDEA 2: GestionaCLM**

*Problema o necesidad*

Las personas que necesitan solicitar ayudas sociales muchas veces tienen dificultades para encontrar la información, saber qué requisitos deben cumplir, pedir una cita o consultar el estado de su solicitud. Además, los trabajadores sociales necesitan una herramienta que les permita organizar los expedientes, documentos y seguimientos de los usuarios.

*Usuarios destinatarios*

- Ciudadanos que solicitan ayudas sociales.
- Trabajadores sociales.
- Técnicos de la Administración.
- Personal encargado de revisar las solicitudes.
- Administradores de la plataforma.

*Solución propuesta*

GestionaCLM sería una aplicación web para gestionar ayudas y servicios sociales de Castilla-La Mancha. La aplicación tendría una parte para los ciudadanos y otra para los profesionales.

Los ciudadanos podrían consultar las ayudas disponibles, enviar solicitudes, adjuntar documentos, pedir citas y consultar el estado de sus expedientes. Los profesionales podrían revisar las solicitudes, añadir informes y cambiar el estado de cada expediente.

Para el proyecto se utilizaría un sistema de usuario y contraseña. La conexión real con sistemas como Cl@ve o el certificado digital se podría dejar como una mejora futura, porque sería demasiado compleja para realizarla completamente.

*Funcionalidades principales*

- Registro e inicio de sesión: La aplicación distinguirá entre ciudadanos, profesionales y administradores. Cada tipo de usuario tendrá diferentes permisos.
- Consulta de ayudas disponibles: Los ciudadanos podrán ver las ayudas disponibles, los requisitos, la documentación necesaria y el plazo para solicitarlas.
- Solicitud online de ayudas: Se podrá rellenar un formulario, adjuntar documentos y enviar la solicitud a través de la plataforma.
- Gestión de expedientes: Los profesionales podrán consultar los expedientes, cambiar su estado, añadir observaciones y registrar las actuaciones realizadas.
- Solicitud de citas: Los ciudadanos podrán pedir una cita con los servicios sociales y consultar la fecha, la hora y el profesional asignado.
- Notificaciones y seguimiento: El usuario podrá recibir avisos sobre cambios en su expediente, documentos que falten, próximas citas o resoluciones.

*Tecnologías que se podrían utilizar*

- HTML, CSS y JavaScript.
- React o Vue para crear una interfaz más dinámica.
- Java con Spring Boot o Node.js con Express para el backend.
- MySQL o PostgreSQL para la base de datos.
- API REST para comunicar el frontend con el backend.
- JSON Web Tokens para controlar las sesiones.
- Bootstap o Material UI para el diseño.
- Git y GitHub para el control de versiones.

**IDEA 3: LegalFlow**

*Problema o necesidad*

Los pequeños despachos de abogados suelen utilizar diferentes programas, correos electrónicos y carpetas para organizar la información de sus clientes, los documentos y los plazos. Esto puede hacer que pierdan mucho tiempo y que exista el riesgo de olvidar fechas importantes o de no informar correctamente al cliente.

*Usuarios destinatarios*

- Abogados autónomos.
- Pequeños despachos de abogados.
- Personal administrativo de los despachos.
- Clientes de los abogados.

*Solución propuesta*

LegalFlow sería una aplicación web para organizar clientes, expedientes, documentos, tareas y fechas importantes. Cada abogado tendría un panel desde el que podría controlar sus casos y comunicarse con sus clientes.

También tendría un portal para los clientes, desde el que podrían consultar el estado de sus expedientes, descargar documentos y enviar archivos al despacho.

El sistema de generación de documentos se basaría inicialmente en plantillas rellenables. De esta forma, el abogado podría crear documentos personalizados sin tener que escribirlos desde cero. La aplicación serviría como ayuda, pero las decisiones y la revisión final siempre dependerían del abogado.

*Funcionalidades principales*

- Gestión de clientes y expedientes: Se podrán registrar clientes, crear expedientes y relacionar cada caso con un área del derecho y un estado.
- Control de plazos y tareas: El abogado podrá registrar fechas de notificaciones, reuniones, vencimientos y tareas pendientes. Todo se podrá consultar desde un calendario.
- Generación de documentos: Se podrán utilizar plantillas de contratos, escritos u otros documentos. El abogado rellenará los datos necesarios y la aplicación generará el documento.
- Portal del cliente: El cliente podrá consultar el estado de sus casos, ver las próximas fechas, descargar documentos y subir nuevos archivos.
- Mensajería y avisos: El abogado y el cliente podrán comunicarse mediante mensajes relacionados con cada expediente. También habrá avisos cuando se reciba un mensaje o un documento nuevo.
- Presupuestos y facturas: El despacho podrá crear presupuestos, registrar servicios y generar facturas de prueba. En el futuro se podría adaptar el sistema a los requisitos de VeriFactu.

*Tecnologías que se podrían utilizar*

- HTML, CSS y JavaScript.

- React o Angular para el frontend.
- Java con Spring Boot para el backend.

- MySQL o PostgreSQL para la base de datos.
- API REST para conectar las diferentes partes de la aplicación.
- Spring Security y JWT para el inicio de sesión.
- Una biblioteca como OpenPDF para generar documentos PDF.
- Sistema de almacenamiento de archivos.
- Git y GitHub para el control de versiones.